



Experiment no.9

Aim : Create a Jenkins CICD Pipeline with SonarQube to perform a static analysis of the code to detect bugs, code smells, and security vulnerabilities on a sample Web / Java / Python application.

Theory :

1. Introduction to CI/CD:

Continuous Integration (CI) and **Continuous Deployment (CD)** are key practices of modern **DevOps** workflows.

They help automate the process of:

- Building code
- Running tests
- Analyzing quality
- Deploying applications efficiently and consistently

Jenkins is one of the most popular open-source automation servers used to build CI/CD pipelines.

It helps integrate multiple tools like **SonarQube**, **Git**, **Maven**, **Docker**, and **Kubernetes** to automate the software development lifecycle.

2. Continuous Integration (CI):

Continuous Integration focuses on **merging code frequently** into a shared repository, where each integration triggers an **automated build and test** process.

This helps detect integration errors early and ensures that the codebase remains stable.

3. Continuous Deployment (CD):

Continuous Deployment automates the **release process** by automatically deploying verified builds to staging or production environments, reducing manual effort and human error.



4. Introduction to Jenkins:

Jenkins is an open-source CI/CD automation tool written in Java. It provides:

- **Automation** for building, testing, and deploying code
 - **Integration** with hundreds of plugins (like SonarQube, Git, Docker, etc.)
 - **Pipeline-as-Code** feature through **Jenkinsfile**
 - **Web interface** for managing jobs and monitoring builds
-

5. Introduction to SonarQube:

SonarQube is an open-source platform used for **static code analysis**.

It helps developers identify:

- **Bugs** – Logic or functional errors
- **Code Smells** – Maintainability or readability issues
- **Security Vulnerabilities** – Potential threats or unsafe coding practices

SonarQube supports multiple languages such as **Java, Python, JavaScript, C++, PHP**, etc., and provides a **dashboard** to visualize metrics like code coverage, duplications, and maintainability.

6. Static Code Analysis (SAST):

Static Application Security Testing (SAST) is the process of analyzing source code **without executing** it to detect:

- Security flaws
- Bad coding practices
- Performance and reliability issues

SonarQube performs SAST by scanning the codebase and reporting:

- **Bugs** → Functional errors
 - **Code Smells** → Poor design/maintenance issues
 - **Vulnerabilities** → Potential security risks
-



VIDYAVARDHINI'S COLLEGE OF ENGINEERING & TECHNOLOGY
DEPARTMENT OF INFORMATION TECHNOLOGY

K.T. Marg, Vasai Road (W), Dist-Palghar - 401202, Maharashtra

7. Jenkins–SonarQube Integration:

Integrating SonarQube with Jenkins enables **automated static code analysis** during every CI/CD build.

This ensures that only high-quality and secure code moves forward in the pipeline.

8. Advantages of Jenkins–SonarQube Integration:

- Ensures **code quality and security** automatically
- Detects bugs and vulnerabilities **before deployment**
- Enforces **coding standards and best practices**
- Provides **visual reports** and metrics for management
- Supports **multi-language projects** in one platform

Procedure:

1) Create a local project on the localhost dashboard



VIDYAVARDHINI'S COLLEGE OF ENGINEERING & TECHNOLOGY

DEPARTMENT OF INFORMATION TECHNOLOGY

K.T. Marg, Vasai Road (W), Dist-Palghar - 401202, Maharashtra

2) Create a folder with the same name as project name in Cdrive and save the sample code to test

Source Code Management

☐ None

☒ Git ?

Repositories ?

Repository URL ?

Please enter Git repository.

Credentials ?

Branches to build ?

Branch Specifier (blank for 'any') ?

Dashboard > retail-store-jenkin-project > Configuration

Configure

- General
- Source Code Management
- Build Triggers
- Build Environment**
- Build Steps
- Post-build Actions

☐ Use secret text(s) or file(s) ?

☐ Add timestamps to the Console Output

☐ Inspect build log for published build scans

☒ Prepare SonarQube Scanner environment ?

Server authentication token

SonarQube authentication token. Mandatory when anonymous access is disabled. Will default to the one defined in the SonarQube installation.

Cannot find any credentials with id retail-token

☐ Terminate a build if it's stuck

☐ With Ant ?



VIDYAVARDHINI'S COLLEGE OF ENGINEERING & TECHNOLOGY
DEPARTMENT OF INFORMATION TECHNOLOGY
K.T. Marg, Vasai Road (W), Dist-Palghar - 401202, Maharashtra

3) Navigate to the same folder on cmd and run

Execute SonarQube Scanner

Task to run ?

JDK ?
JDK to be used for this SonarQube analysis
(Inherit From Job)

Path to project properties ?

Analysis properties ?
sonar.projectKey=retail-store-jenkins-project
sonar.projectName=retail-store-jenkins-project
sonar.projectVersion=1.0
sonar.language=java
sonar.tests=src/test/java
sonar.sources=src/main/java

Additional arguments ?

Dashboard > retail-store-jenkins-project > #1 > Console Output

```
INFO: Sensor Zero Coverage Sensor
INFO: Sensor Zero Coverage Sensor (done) | time=3ms
INFO: Sensor Java CPD Block Indexer
INFO: Sensor Java CPD Block Indexer (done) | time=4ms
INFO: SCM Publisher SCM provider for this project is: git
INFO: SCM Publisher 18 source files to be analyzed
INFO: SCM Publisher 18/18 source files have been analyzed (done) | time=66ms
INFO: CPD Executor 7 files had no CPD blocks
INFO: CPD Executor Calculating CPD for 4 files
INFO: CPD Executor CPD calculation finished (done) | time=3ms
INFO: Analysis report generated in 24ms, dir size=151.9 kB
INFO: Analysis report compressed in 19ms, zip size=40.1 kB
INFO: Analysis report uploaded in 38ms
INFO: ANALYSIS SUCCESSFUL, you can find the results at: http://192.168.1.33:9000/dashboard?id=retail-store-jenkins-project
INFO: Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report
INFO: More about the report processing at http://192.168.1.33:9000/api/ci/task?id=AYpw8e9ul2YFw-1Mxqd1
INFO: Analysis total time: 3.444 s
INFO: -----
INFO: EXECUTION SUCCESS
INFO: -----
INFO: Total time: 4.117s
INFO: Final Memory: 27M/94M
INFO: -----
Finished: SUCCESS
```



VIDYAVARDHINI'S COLLEGE OF ENGINEERING & TECHNOLOGY
DEPARTMENT OF INFORMATION TECHNOLOGY
K.T. Marg, Vasai Road (W), Dist-Palghar - 401202, Maharashtra

4) Report will be available on the Dashboard.

The screenshot shows the SonarQube Overview dashboard for the 'retail-store-jenkins-project'. The dashboard includes a navigation bar with tabs for Overview, Issues, Security Hotspots, Measures, Code, and Activity. A central message says 'Enjoy your sparkling clean code!'. The dashboard displays various quality metrics:

- Bugs:** 0 (A)
- Code Smells:** 1 (A)
- Security:** 0 Vulnerabilities (A)
- Security Review:** 0 Security Hotspots (A)
- Coverage:** 0.0% Coverage (Coverage on 26 Lines to cover) (Unit Tests)
- Duplications:** 0.0% Duplications (Duplications on 124 Lines) (0 Duplicated Blocks)

The Activity section at the bottom shows a list of recent activities.

The screenshot shows the SonarQube Issues page for the 'retail-store-jenkins-project'. The page displays a list of issues, with one issue selected:

- Issue:** Make this interface functional or replace it with a function type. (Code Smell)
- Location:** src/.../strategy/DiscountStrategy.kt
- Severity:** Code Smell
- Message:** Make this interface functional or replace it with a function type.
- Effort:** 5min effort · 3 months ago

The issue details section shows the code snippet where the issue was found:

```
7 fun calculateNetPayableAmount(bill: Bill): Double
8
9 companion object {
10     const val DISCOUNT_PER_100 = 5.0
11 }
12
13
```

The screenshot shows the SonarQube Project Summary for the 'retail-store-jenkins-project'. The project status is 'Passed' and the last analysis was 2 minutes ago. The summary displays the following metrics:

Metric	Value	Quality Gate
Bugs	0	A
Vulnerabilities	0	A
Hotspots Reviewed	—	A
Code Smells	1	A
Coverage	0.0%	
Duplications	0.0%	
Lines	124	XS

The project is configured for Kotlin and Java.



VIDYAVARDHINI'S COLLEGE OF ENGINEERING & TECHNOLOGY
DEPARTMENT OF INFORMATION TECHNOLOGY

K.T. Marg, Vasai Road (W), Dist-Palghar - 401202, Maharashtra

Conclusion :

Integrating SonarQube with Jenkins CI/CD enables continuous and automated static analysis of source code.

This helps developers maintain high code quality, detect security vulnerabilities early, and ensure that only clean, secure, and production-ready code is deployed.

Such automation forms a critical part of DevSecOps, embedding security and quality checks directly into the development pipeline.